Islamic University of Technology

Lab 9: Functions

CSE 4108 : Structured Programming Lab

Section 1B

MIA, NFS, IAC, ASH

19 November 2024

1 Lab Tasks

1.1 Kinda Average

Write a program that utilizes a function to calculate the average of an array and returns it. The declaration of the function is like below.

```
double calculateAverage(double arr[], int len);
```

Design the input and output accordingly to match the task.

1.2 Transposed Again

Write a C program that would utilize a function to perform the transpose of a 4×4 matrix. The function would take two 2D arrays as inputs. Since array elements are modified in place in C, the function would store the transpose of the first matrix into the second one. 4×4 size is fixed. The declaration of the function looks like below.

```
double getTranspose(double inMat[][4], double transposedMat[][4]);
```

Design the input and output accordingly to match the task.

1.3 Padovan Plastic (RECURSION)

The Padovan sequence is a fascinating number sequence that appears in various contexts in mathematics, including number theory, geometry, and even in the analysis of natural phenomena. This sequence grows asymptotically as a power of the plastic number ($\rho \approx 1.3247$), which is the unique real solution to the equation $x^3 = x + 1$. That means the ratio of consecutive terms in the Padovan sequence approaches the plastic number.

The Padovan sequence is a number sequence in which each term is the sum of the two preceding terms, but with an offset compared to the Fibonacci sequence. Specifically, the sequence is defined by the recurrence relation:

$$P(n) = P(n-2) + P(n-3)$$

The sequence starts, $1, 1, 1, 2, 2, 3, 4, 5, 7, 9, ...$

Now, write a program utilizing a function that **recursively** calculates the nth Padovan term (P_n). The declaration of the function is like below.

```
int getPadovanAt(int n);
```

[Sample Program]
Input

```
Enter term, n: 7
```

Output

```
P7 : 5
```

[Hint] *Use the sequence to get the terminating condition for the recursion*

1.4 Help Yourself

For the past few labs, you have been so tirelessly reading strings character by character and lastly putting them in a character array. Does is not feel boring? Doing the same thing again and again and again ... Help yourself!

Write the following function that would take a **sufficiently large character array** as input, keep reading and populating one character by one character until an ending/delimiting character, like *newline* is found and then return the number of characters read.

```
1 int readSentence(char sentence[], char delim); // returns length of sentence, delim
      can be set to '\n' while calling
```

To test out your program, you can try to print out the length of the sentence read and then the sentence itself.

1.5 Over 9000! (RECURSION)

Calculation of power can be made faster by calculating x^n in a different way. We first notice that if n is a power of 2, then x^n can be computed by squaring. For example, x^4 is the square of x^2, so x^4 can be computed using only two multiplications instead of three. As it happens, this technique can be used even when n is not power of 2. If n is even, we use the formula $x^n = (x^{n/2})^2$. If n is odd, then $x^n = x \times x^{n-1}$. Write a **recursive function** that computes x^n. (The recursion ends when $n = 0$, in which case the function returns 1). To test your function, write a program that asks the user to enter values for x and n, calls power to compute x^n, and then displays the value returned by the recursive function. The declaration of the function looks like below.

```
1 long long power(long long x, int n);
```